Copyright All rights reserved

13. 함수 포인터
(union)

Heesung Oh
https://github.com/3dapi

13.1 함수 포인터 기초

Heesung Oh
Copyright All rights reserved

- 함수
  - 코드 영역에 존재, 고유한 주소 가짐
  - 함수 이름: 함수의 시작 주소로 변환되어 사용
- 함수 주소
  - 변수에 저장 가능
  - 저장된 주소 해당 함수 호출 가능
- 함수 포인터
  - 함수의 시작 주소 저장 변수

```
#include <stdio.h>

int Add(int a, int b)
{ return a + b; }

int main(void)
{
  printf("%p\n", Add);
  printf("%p\n", &Add);
  return 0;
}
```

13.1 함수 포인터 활용

Heesung Oh
Copyright All rights reserved

- **함수 포인터**
  - 함수의 시작 주소 저장 변수 → 코드 영역의 함수 위치
  - 함수 주소 전달 가능, 실행할 함수 상황에 따라 선택 가능
- **구조적 효과**
  - 조건문 감소, 코드 수정 최소화, 흐름 유연성 증가
- **활용**
  - 콜백, 함수 테이블, 상태 머신, 이벤트 처리

```
int (*fp)(int, int);     // 함수 포인터 선언

fp = Add;                // 함수 주소 저장
int result = fp(10, 20);  // 함수 호출

// 또는
int result = (*fp)(10, 20); // 동일한 의미
```

13.1.1 함수 포인터 선언

Heesung Oh
Copyright All rights reserved

- **함수 포인터**
  - 함수의 주소를 담는 변수. 반환형 + 매개변수 형태 → 반드시 일치

- **함수 포인터 선언**

```
함수의 반환 (*함수_포인터_이름)(함수_인수_ 타입_리스트);

// 이름이 없는 형태
함수의 반환 (*)(함수_인수_ 타입_리스트);
```

- **구조 해석**

```
int (*fp)(int, int);
```

  - *fp → 포인터 변수
  - (int, int) → 정수 두 개 매개변수
  - int → 반환형
  - → fp : 정수 두 개를 받아 정수를 반환하는 함수의 주소 저장

13.1.1 함수 포인터 선언

Heesung Oh
Copyright All rights reserved

- 함수 포인터

```
int (*fp)(int, int);
```

- 함수 선언

```
int* fp(int, int);
```

13.1.2 함수 포인터 초기화와 호출

Heesung Oh
Copyright All rights reserved

- 함수 이름
  - 표현식에서 함수에 대한 포인터로 자동 변환

```
fp = Add;        // &Add 으로 변환,
                 // 함수 포인터에 저장
```

- 함수 포인터로 함수 호출 가능
  - fp(10, 20);

- 명시적 형태
  - (*fp)(10, 20); // 동일한 의미 fp() == (*fp)()

13.1.2 함수 포인터 초기화와 호출

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int Add(int a, int b) { return a + b; }
int Mul(int a, int b) { return a * b; }

int main(void)
{
    int (*fp)(int, int);
    fp = Add;
    printf("Add 결과: %d\n", fp(10, 20));
    fp = Mul;
    printf("Mul 결과: %d\n", fp(10, 20));
    return 0;
}
```

13.1.3 typedef로 함수 포인터 단순화

Heesung Oh
Copyright All rights reserved

- typedef
  - 함수 포인터 단순화 → 가독성 향상
  - 타입 이름 → 의미 전달
  - 함수 포인터 → 개념 단위로 사용
  - 코드 구조 → 명확 / 확장 용이

```
typedef int (*BinaryOp)(int, int);
int Add(int a, int b) { return a + b; }
int Sub(int a, int b) { return a - b; }

int main(void) {
    BinaryOp op = Add;            // 함수 포인터에 함수 대입
    printf("%d\n", op(10, 20));   // Add 호출
    op = Sub;
    printf("%d\n", op(10, 20));   // Sub 호출
}
```

```
typedef void (*OnEvent)(int eventCode, void* userData);

// OnEvent 타입으로 정의된 함수 포인터를 전달받음
void RegisterCallback(OnEvent callback);
```

13.1.4 함수 포인터 배열

Heesung Oh
Copyright All rights reserved

- **함수 포인터 배열**
  - 동일한 형태의 함수들을 배열에 저장
    → 인덱스로 함수 선택 후 호출
    → 조건문 없이 실행 흐름 제어
  - 배열 요소 = 함수
    인덱스 = 선택 기준
  - 선택 구조 단순화, 가독성 향상, 유지보수 용이, 함수 추가 시 배열만 수정
- **구성**
  - 함수 포인터 배열 = 함수 테이블
  - 실행 흐름을 데이터(인덱스)로 제어
- **활용 예**
  - 행동/기술 테이블
  - 메뉴 처리, 상태 분기, 이벤트 처리

13.1.4 함수 포인터 배열

Heesung Oh
Copyright All rights reserved

```c
typedef int(*OP)(int, int);

int Add(int a, int b) { return a + b; }
int Sub(int a, int b) { return a - b; }
int Mul(int a, int b) { return a * b; }
int Div(int a, int b) {
    if(b == 0) {
        printf("0으로 나눌 수 없습니다.\n");
        return 0;
    }
    return a / b;
}
```

```c
int main(void)
{
    int a = 20;
    int b = 10;
    int choice;

    // 0: Add, 1: Sub, 2: Mul, 3: Div
    int(*ops[])(int, int) = {Add, Sub, Mul, Div};
    //OP ops[] = {Add, Sub, Mul, Div};

    printf("연산 선택 (0: +, 1: -, 2: *, 3: /) : ");
    scanf("%d", &choice);

    if(choice < 0 || choice > 3) {
        printf("잘못된 선택입니다.\n");
        return 0;
    }

    printf("결과: %d\n", ops[choice](a, b));
    return 0;
}
```

13.2.1 콜백 함수와 qsort

Heesung Oh
Copyright All rights reserved

- 콜백 함수 (Callback Function)
  - 평소 대기 → 이벤트 발생 시 호출
  - 다른 코드가 대신 호출하는 함수
  - 즉, 함수의 주소를 다른 코드에 넘겨주고 필요할 때 대신 호출되는 함수
- 특징
  - 호출 시점 제어 → 외부
  - 코드 분리
- 일반 호출 vs 콜백 구조
  - 일반호출: 내 코드 → 내가 함수 호출
  - 콜백 구조: 내 코드 → 함수 주소 전달 → 외부 코드 → 필요 시 호출
- 활용
  - 버튼 클릭 처리
  - 충돌 처리
  - 네트워크 이벤트 처리

13.2.1 qsort

Heesung Oh
Copyright All rights reserved

- qsort
  (base, count, size, compare)
  - 배열 정렬 함수
  - 자료형 독립 (void*)
  - 정렬 알고리즘 → qsort
  - 정렬 기준 → 사용자 함수 (콜백)
  - 일반적으로 O(n log n) 성능
- 특징
  - 빠른 평균 성능
  - 대용량 데이터 처리 적합
  - 추가 메모리 거의 없음 (in-place)
  - 안정 정렬 아님 (stable 보장 없음)
- 매개변수
  - base : 배열 시작 주소
  - count : 요소 개수
  - size : 요소 크기 (byte)
  - compare : 비교 함수 (콜백)
- 비교 함수
  - 두 요소 비교 수행
  - 반환값
  - 음수 : a < b
  - 0 : a == b
  - 양수 : a > b

13.2.1 qsort

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>
#include <stdlib.h>

int CompareInt(const void* a, const void* b)
{
    int lhs = *(const int*)a;
    int rhs = *(const int*)b;

    if (lhs < rhs)
        return -1;
    if (lhs > rhs)
        return  1;
    return 0;
}

int main(void)
{
    int arr[] = { 42, 7, 19, 3, 25 };
    int size = sizeof(arr) / sizeof(arr[0]);

    qsort(arr, size, sizeof(int), CompareInt);

    for (int i = 0; i < size; ++i)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

```c
int ComparePlayer(const void* p1, const void* p2) {
const Player* lhsPlayer = (const Player*)p1;
const Player* rhsPlayer = (const Player*)p2;

    if(lhsPlayer->level > rhsPlayer->level)
        return -1;
    if(lhsPlayer->level < rhsPlayer->level)
        return 1;
    return 0;
}

int main(void)
{
    // ...
    // sort by level
    qsort(player, size, sizeof(Player)
                , ComparePlayerLevel);

    // ...
    return 0;
}
```

13.2.2 함수 포인터와 void* 응용

Heesung Oh
Copyright All rights reserved

- 콜백 + userData (void*)
  - 콜백 호출 시 같은 userData 전달
  - 함수 호출과 데이터 전달 분리 → 자료형 독립적인 공용 경로 구성
  - void* → 실제 타입 정보 없음 → 어떤 데이터의 주소 든 전달 가능
- 사용
  - 콜백 내부에서 원래 타입으로 캐스팅
- 주의
  - 캐스팅 책임 → 콜백 작성자
- 활용
  - 구조체 포인터 전달
  - 버튼 정보
  - 윈도우 핸들
  - 게임 오브젝트

13.2.2 함수 포인터와 void* 응용

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

typedef void (*Callback)(void*);

void InvokeThreeTimes(Callback cb, void* userData)
{
    for (int i = 0; i < 3; ++i)
        cb(userData);   // 매번 같은 userData 전달
}

void PrintMessage(void* userData)
{
    const char* msg = (const char*)userData;
    printf("%s\n", msg);
}

int main(void)
{
    const char* text = "안녕하세요, 콜백입니다.";
    InvokeThreeTimes(PrintMessage, (void*)text);
    return 0;
}
```

13.2.3 함수 포인터와 const 결합

Heesung Oh
Copyright All rights reserved

- const fp
  - 포인터 변수 자체 고정
  - 함수 호출은 정상적으로 가능
- 주의
  - 함수 자체는 수정 대상 아님
  - 함수에 const 적용 의미 없음
- 활용
  - 디스패치 테이블
  - 함수 테이블 고정
  - 초기화 이후 변경 방지

```
int (*fp)(int, int);
int (*const fp)(int, int);

fp       → 가리키는 함수 변경 가능
const fp → 한 번 정하면 변경 불가
```

13.2.4 함수 포인터와 구조체 결합

Heesung Oh
Copyright All rights reserved

- 구조체 + 함수 포인터
  - 데이터 + 동작 함께 구성
  - 각 객체가 자신의 동작을 직접 보유
  - C에서도 객체처럼 동작하는 구조 구성 가능

- 구조
  - 구조체 → 데이터
  - 함수 포인터 → 동작

- 예
  - Button
    - text → 표시 정보
    - onClick → 실행 함수
  - Actor 구조
    - update / draw / 충돌 처리 함수 포함

13.2.5 함수 포인터를 이용한 상태 머신 구현

Heesung Oh
Copyright All rights reserved

- 상태 표현 방식
  - enum + switch
  - 외부에서 상태 분기 → 상태가 늘어나면 분기 코드가 복잡해짐

- 함수 포인터 방식
  - 현재 상태 함수 호출 → 상태 함수 내부에서 다음 상태 결정
  - 상태 = 함수 포인터, 전환 = 포인터 변경
  - 현재 상태 함수 하나만 호출
  - 상태 수 증가해도 구조 유지

13.2.5 함수 포인터를 이용한 상태 머신 구현

Heesung Oh
Copyright All rights reserved

```c
typedef enum {
    STATE_IDLE,
    STATE_MOVE,
    STATE_ATTACK
} State;

State state = STATE_IDLE;

/* ... */

switch (state) {
case STATE_IDLE:   /* ... */ break;
case STATE_MOVE:   /* ... */ break;
case STATE_ATTACK: /* ... */ break;
}
```

```c
#include <stdio.h>

void Idle(void);
void Move(void);
void Attack(void);
void (*currentState)(void) = Idle;

void Idle(void)
{
    printf("Idle 상태\n");
    currentState = Move;   // 예시로 다음 상태로 변경
}
void Move(void)
{
    printf("Move 상태\n");
    currentState = Attack;
}
void Attack(void)
{
    printf("Attack 상태\n");
    currentState = Idle;
}

int main(void)
{
    for (int i = 0; i < 5; ++i) {
        currentState();     // 현재 상태의 함수 호출
    }
    return 0;
}
```

13.2.6 함수 포인터와 플랫폼 API

Heesung Oh
Copyright All rights reserved

- Windows API
  - WndProc → OS가 호출
  - 함수 포인터 + 호출 규약 → 함수 형태 + 호출 방식 일치 필요
  - 콜백 구조: 함수 주소 등록 → 필요 시 호출
  - 호출 규약: 누가 스택을 정리하는지에 대한 약속

호출 규약	스택 정리 주체	인자 전달 방식	특징
__cdecl	Caller (호출한 쪽에서 정리)	스택, 오른쪽 → 왼쪽	일반
__stdcall	Callee (함수 내부에서 정리)	스택, 오른쪽 → 왼쪽	Windows API

13.2.7 최신 C 표준과 함수 포인터

Heesung Oh
Copyright All rights reserved

- 함수 포인터
  - C 초기부터 문법 / 의미 변화 없음
- 문제
  - 스레드 / 원자 연산 / 메모리 모델 (동시성)
  - → 잘못된 값 읽기 → 잘못된 함수 호출
- 해결
  - _Atomic → 안전한 읽기 / 쓰기 보장
  - ※ 함수 포인터는 그대로 유지

```c
#include <stdatomic.h>

int Add(int a, int b) { return a + b; }
int Sub(int a, int b) { return a - b; }

_Atomic(int (*)(int, int)) op = Add;

void SetOp(int (*newOp)(int, int))
{
    atomic_store(&op, newOp);
}

int CallOp(int a, int b)
{
    int (*f)(int, int) = atomic_load(&op);
    return f(a, b);
}
```

Heesung Oh
Copyright All rights reserved

- 함수 포인터
  - 함수의 주소를 저장하는 포인터
  - 함수를 값처럼 전달 / 호출 가능 → 흐름 제어
  - 콜백 / 함수 테이블 / 상태 머신 구현 가능
  - 함수 형태 일치 필요 (반환형, 매개변수)
  - 호출 규약 포함한 형태 일치 필요
  - 구조체 결합 → 객체처럼 사용
  - void* 결합 → 데이터 전달 분리
  - 문법 변화 없음 (C 초기부터 동일): 멀티 스레드 환경 → _Atomic 필요

실습 및 과제

Heesung Oh
Copyright All rights reserved

- 과제1
  - 사칙연산 함수 포인터 테이블
  - 연산 기호 입력 → 함수 선택 → 실행
- 과제2
  - qsort + 콜백
  - 비교 함수 변경 → 정렬 기준 변경
- 과제3
  - 상태 머신
  - 상태 = 함수 포인터
  - currentState() 반복 호출
- 과제4
  - Button 구조체
  - onClick 함수 포인터
  - 클릭 → 함수 실행